Paisave: Security Self-Assessment Report

2026 FinTechathon International Track, Topic A (Personal Finance Assistant). Team ACE 2.0

1. Purpose and scope

This report answers the submission checklist item *Security self-assessment: permission-tier implementation and known-risk list*. It covers the Paisave application as submitted: the web app (PWA), the API and tool servers, the worker, the agent runtime and gateway, the PostgreSQL and Dragonfly stores, and the models.

It describes what is implemented in the source code, what we verified, and what is still open. Where something is a plan and not a fact, we say so.

System in one paragraph. Paisave is an AI agent that helps a person in Pakistan budget and save. It reads bank-alert emails through a read-only Gmail connection, turns them into a categorised ledger, splits income across goals, and matches card discounts to what the user actually spends. Agents act only by calling typed tools that read and write the user's own records in our database. **No component can start a payment, transfer or any other write to a bank.**

2. Security design principles

1. **Least authority.** The worst thing a compromised or confused agent can do is write a wrong number in one user's ledger. There is no payment tool to misuse.
2. **Enforce in the database, not only in the API.** Per-user isolation is a Postgres row-level security policy, so an application bug cannot widen access.
3. **Keep the money maths out of the LLM.** Goal splits, savings rules, discount value and deal redemption are plain tested code. The LLM handles language, parsing fallbacks and judgement calls only.
4. **Private data stays on our own model.** Only public-data agents may use an optional third-party model, and a fail-closed screen sits in front of them.
5. **Minimise what is collected.** No card digits, CVV, PIN or OTP. Gmail access is read-only and only bank-alert messages are parsed.
6. **Fail closed.** If the screening model is offline, new merchants stay hidden. If a key is unknown, the call is refused and delayed.

3. Permission tiers as implemented

Tier	Principal	Granted	Denied	Where enforced
T0 Public	Anonymous visitor	Sign up (emailed one-time code), sign in, password reset	Everything else	`api/auth.py`, rate limits

Tier	Principal	Granted	Denied	Where enforced
T1 User	Signed-in person	Read and write **own rows only**; chat with own agents	Other users' data, admin pages	`core/deps.py` (`user_principal`, `own_db`), Postgres RLS
T2 User agents	Per-user team of 3 to 4 bots	`fin_user` tools (about 32 of the 72 the runtime offers) with the room's member key	Create bots or rooms, run commands, read other chats, move money	`ai-ml/gateway/teams.py` allow-lists, `mcp/user_server.py`
T3 Shared desk agents	29 market and deal bots	`fin_market` tools over public data and shared tables	Any user id, balance or transaction	Allow-lists, tool design, merchant screen
T4 Admin	Separate admin accounts (no finances, no chat)	Manage users, deals, cards, agents, plans; every change is audited	Act as a user; use user routes	`require_admin`, `audit_log`
T5 System	Worker and gateway	Background jobs, migrations, agent provisioning	Reachable from the internet	Shared internal key (constant-time compared), private network

3.1 How a request is fenced in

1. The access token is verified and must be of type `access`.
2. The server-side session is checked: **active**, **locked** (idle timeout, API answers 423) or **missing**.
3. A principal (user id and role) is built. Admin accounts are refused on user routes and users are refused on admin routes.
4. A scoped database session sets `app.user_id` and `app.role` as the first statement of the transaction.
5. PostgreSQL applies `FORCE ROW LEVEL SECURITY` policies on every user-owned table. `FORCE` binds the table owner too.

The policy on each user-owned table has the form `user_id::text = current_setting('app.user_id', true) OR current_setting('app.role', true) IN ('admin','system')`. Shared reference tables are readable by everyone and writable only by admin or system roles. Two private tables (`agent_registry`, `audit_log`) are staff-only.

3.2 How agents are tied to one user

The agent runtime sends no caller identity to tool servers. Each user's room therefore holds a random **member key** that resolves to exactly one user. Keys are stored hashed and encrypted. A tool call with a valid key opens a scoped session for that user, so the same row-level policy applies. Tool calls are rate-limited (600 per minute per key), and unknown keys are throttled and delayed.

4. Controls in place

Area	Control
Authentication	Argon2 password hashing; short-lived signed access token plus a rotating refresh token held in a server-side session; emailed one-time code on sign-up; passkeys (fingerprint or face) through WebAuthn.

Area	Control
Session safety	Idle lock like a banking app: after inactivity the session is locked until the password or a passkey is used again. Unlock attempts are limited (5 per 15 minutes). Sessions can be revoked individually or all at once.
Browser protections	`HttpOnly` and `SameSite=Lax` cookies, `Secure` in production. Every cookie-authenticated write must carry a custom header that cross-site forms cannot send, otherwise the API answers 403 (CSRF guard). CORS is limited to the configured front-end origin.
Abuse limits	Rate limits on sign-up (10 per hour per IP), one-time-code verify and resend, sign-in (8 per 15 minutes per IP and email), password reset, unlock and tool calls.
Data minimisation	No card numbers, CVV, PIN or OTP are requested or stored; only network, tier and debit or credit. A bank account keeps an optional last-4 from alerts to match them. Of each Gmail message only the id and subject are kept, never the body.
Gmail access	`gmail.readonly` scope only. Only messages from the alert domains of banks the user named are fetched. Access and refresh tokens are encrypted at rest (Fernet).
Model boundary	User chats, advisors, bank-email parsing and merchant screening use only the self-hosted model. Shared desks may use a third-party model because they see public data only.
Merchant screen	Each new payee is labelled business or person by the self-hosted model. Persons are marked `personal` and never shown to shared bots. If the model is offline, the merchant stays hidden.
Secrets	One git-ignored root `.env`. The agent container receives only 16 of the variables, so JWT, encryption, SMTP, OAuth and database-owner secrets never reach it.
Auditability	Admin actions (user update or delete, deal edits, agent sync) are written to `audit_log`. Each ledger entry records its source (`gmail`, `chat`, `deal`).
Responsible AI	The agent states it is not a licensed financial advisor and gives no guarantees. The optional investment board needs the user's explicit agreement to a disclaimer; the tool refuses without the acceptance flag and records the time. The flag is set by the agent after showing the disclaimer, so it relies on the agent following its instructions.
Deletion	Deleting a user removes their rows by cascade and their agents and room in the runtime. Failed runtime clean-ups are retried on later syncs.

5. Data inventory

Data	Where	Sensitivity	Protection
Email, name, password hash	`users`	Personal	Argon2 hash; RLS
Bank, wallet and card *type*	`bank_accounts`, `cards`	Financial	RLS; no card numbers

Data	Where	Sensitivity	Protection
Transactions, balances, goals	`transactions`, `goals`, `savings_ledger`	Financial	RLS
Gmail tokens	`email_connections`	Credential	Encrypted at rest; read-only scope
Processed email ids and subjects	`ingested_emails`	Personal	RLS; no bodies
Chat history and remembered facts	`chat_messages`, `user_facts`	Personal	RLS; own-model only
Push subscriptions, passkeys	`push_subscriptions`, `webauthn_credentials`	Credential	RLS
Merchants, deals, market data	shared tables	Public	Merchants screened first
Admin actions	`audit_log`	Operational	Staff-only

6. Verification performed

- **Row-level security** was tested directly in PostgreSQL: one user cannot read or write another user's rows.
- **Automated tests (22):** goal-split conservation and caps; alert parsing, merchant names and protection against stale alerts; deal-redemption rules; merchant screening including the offline fail-closed case; card-catalogue enforcement.
- **End-to-end check** with a mock model: sign-up, per-user rooms, a chat message that calls the user tools with that user's key and returns only that user's data, and the reply returning to the chat history.
- **Simulated bank alerts** (the Test bank email button) pass through the real parser, categoriser, savings split and deal detection.

Not yet done: an independent penetration test, a dependency vulnerability scan in CI, measurement of the self-hosted model on a fixed evaluation set, and a full scheduled agent cycle on the production model.

7. Known-risk list

Ratings are our own judgement of residual risk after the mitigation: **High**, **Medium**, **Low**.

#	Risk	Residual	Mitigation now	Planned
R1	**Prompt injection** through email subjects, merchant names or chat text may steer an agent.	Medium	Agents can call only typed, user-scoped tools. No payment tool exists. Money rules live in code. Only subjects of bank-domain emails are read.	Confirmation step for large manual entries; output checks on tool arguments.
R2	**Member key exposure.** The key sits in the agent's prompt and could in theory be repeated in chat.	Medium	A key resolves to the user's own data only, so repetition shows the user their own data. Rate limits and throttling on bad keys. Keys hashed and encrypted at rest.	Per-call signed tokens if the runtime adds caller identity; key rotation.

#	Risk	Residual	Mitigation now	Planned
R3	**Small-model reliability.** Tool calling and onboarding quality of Qwen3.5-9B are not yet measured at scale.	Medium	Tools validate inputs and enforce business rules (for example the card catalogue). Money maths is deterministic.	Evaluation set and published scores.
R4	**Mis-parsed bank alert.** Unknown formats fall back to the LLM and may be wrong.	Medium	Regex first. Stale alerts never override a stated balance. Users can correct by chat.	More parsers; confidence threshold before saving.
R5	**Wrong deal credit.** Merchant-name matching (“KFC” vs “KFC Pakistan”) can miss a use. The debit is assumed to be after the discount.	Low	Shown as “about Rs. X”. Only offered deals are tracked. Credit-card deals stay manual. One redemption per offer and per transaction.	Alias table for merchants.
R6	**Third-party model sees merchant names.** Shared deal bots may run on a third-party model.	Low	Names are screened first; private payees are hidden; the screen fails closed. Users’ balances and ids never reach shared bots.	Option to run all desks on the self-hosted model (already supported by configuration).
R7	**Account takeover.** No multi-factor beyond passkeys; no device binding.	Medium	Argon2, rate limits, idle lock, passkeys, rotating refresh tokens.	Mandatory second factor for new devices.
R8	**Cookie `Secure` flag is off by default** for local development.	Medium in a misconfigured deploy	Production checklist requires `COOKIE_SECURE=true` over HTTPS.	Refuse to start in production with it off.
R9	**Demo accounts and data** exist for evaluation (`0002_demo_data`).	Low	Skipped when `ENVIRONMENT=production`; removal steps documented.	Delete the migration before launch.
R10	**Unpinned runtime image.** The agent runtime image uses `latest`.	Low	Documented in the deployment checklist.	Pin to a release tag.
R11	**Scraped and third-party market data** may be wrong or stale.	Low	Presented as signals with sources and a disclaimer; the investment board is opt-in.	Source freshness checks.
R12	**Scheduled agent runs** are verified only as far as the trigger call.	Low	Admin console shows each agent’s status and last error.	Full-cycle test on the production model.

#	Risk	Residual	Mitigation now	Planned
R13	**Regulatory scope.** The product gives education and tracking, not licensed advice. Wider use needs review by a licensed partner.	Low today	Disclaimers; no guarantees; opt-in investing.	Regulatory review before any advice product.

8. Residual risk summary

No high residual risks remain in the submitted scope, because the product has no authority over real money and enforces user isolation in the database. The medium risks (R1 to R4, R7, R8) are about the quality of the agent's judgement and about account safety. They can degrade the accuracy of a user's own ledger or expose that user's own account if their credentials are stolen, but they cannot move funds or expose other users.

9. Incident and change handling

- **Credential compromise:** an admin can deactivate a user and revoke all sessions; the user can disconnect Gmail, which stops ingestion.
- **Agent misbehaviour:** admins can re-sync or disable agents from the admin console; agent behaviour is defined in versioned prompt files and re-applied only when the files change.
- **Schema changes** go through Alembic migrations, and security policies are created in migrations, not by hand.